
The Kalshi Trading Bot — A Plain-English Tour

What I'm building, why it works, and why it's only possible now

Gabriel Kagan

May 2026

The thirty-second version

I built a robot that trades on a regulated U.S. exchange called Kalshi. Kalshi is a kind of betting market: instead of stocks, you trade contracts on whether something will happen. “Will Bitcoin be above \$65,000 fifteen minutes from now?” If yes, the contract pays \$1. If no, it pays nothing. Most contracts trade between 5¢ and 95¢ before they settle, depending on how likely the market thinks the answer is.

The robot’s job is simple to describe: every fifteen minutes, look at hundreds of these contracts, estimate the real probability of each outcome better than the market does, and place small, careful bets when the price and the probability disagree.

It runs 24 hours a day, by itself, on a small cloud computer. It has been trading real money since February 22, 2026.

What is the bot actually doing?

Imagine you’re standing in front of a market stall that sells little betting slips. Every fifteen minutes, the stall puts out a fresh batch of slips. Each slip says something like: “*Pays \$1 if Bitcoin is above \$65,000 at 2:45 PM.*” You can buy a slip for whatever the crowd is currently paying — let’s say 80¢.

If you think Bitcoin has, say, a 90% chance of being above \$65,000 at 2:45, then an 80¢ slip is a good deal: you’re paying 80¢ for something worth 90¢ on average. Buy it, and over thousands of similar bets, you should make money.

The hard part is the **estimating**. You need to know, more accurately than the crowd, what the real probability is. The bot does this by:

1. **Watching the actual price of Bitcoin** on Coinbase and Kraken, in real time, sub-second.
2. **Measuring how wiggly the price is** — how much it bounces around per minute. (Bouncy prices mean more uncertainty.)
3. **Doing math** to translate the current price, the wiggleness, and the time remaining into a probability number.
4. **Comparing** that probability to what the market is currently charging for the slip.
5. **Buying** the slip if the gap is big enough to be worth the fee.

That’s it. There are many layers of cleverness behind each of those five steps, but that’s the whole shape of the thing.

Why this works at all

Three reasons it’s not just gambling:

It's never tired, never bored, never excited. A human trader looking at Kalshi can check maybe one or two markets at a time, and they get worse when they're tired or rattled by a recent loss. The bot looks at every single live market, every second, with the same discipline forever.

It says no most of the time. The bot examines hundreds of markets per day and trades only the small handful where its math says there's a real edge over the price after fees. Out of every hundred markets it could trade, it probably skips ninety-five. Saying no is most of the job.

It uses small, sensible bet sizes. No bet risks a large fraction of the account. When the account is down, the bot automatically cuts its bet sizes in half, then to quarters, then to a 10% minimum floor if losses pass a steep threshold. (The bot doesn't fully stop on a deep drawdown — small bets keep producing the feedback the system needs to tell a recoverable streak from a real regime change. A true stop is something I do by hand.) A long bad streak hurts but doesn't ruin you.

Where the math actually comes from

Some of the techniques in the bot are things that quant trading firms have used for twenty years — they're just rarely brought to a retail prediction market.

- The way it measures wiggleness ("volatility") uses a method from a 2008 academic paper by Barndorff-Nielsen and three co-authors (Hansen, Lunde, Shephard) that's standard at institutional trading desks.
- The way it predicts how wiggleness changes through the day (it tends to cluster — a calm hour is followed by another calm hour, a wild hour by another wild hour) uses something called an EGARCH model, also standard at desks.
- The way it translates wiggleness into a probability uses a particular bell-curve shape (Normal Inverse Gaussian) that fits crypto returns much better than the simple bell curve most people imagine.
- The way it learns from its own past trades — adjusting its probability estimates when it discovers it has been consistently a few percentage points too confident — is a calibration system that automatically gets better as more trades settle.

None of this is magic. It's just a lot of careful, conservative engineering applied to a kind of market most people haven't thought about systematically yet.

What's live right now

As of May 2026, the bot is actively trading:

- **Six cryptocurrencies** in 15-minute windows: Bitcoin, Ethereum, Solana, XRP, HYPE, and Dogecoin. Each has slightly different rules and price floors based on how well the bot's model has historically worked on that coin. A seventh — BNB — was added on 2026-05-17 in shadow mode: the bot evaluates BNB windows the same way as the live six but does

not place any real BNB orders yet. It's accumulating data so the calibration model can be trained before BNB graduates to live trading (target: around 2026-06-10).

- **Specialty overlays** that look for narrower opportunities: contracts that are almost certainly going to settle one way ("decided contracts"), late-window momentum trades in the final 1–5 minutes before settlement, and weekend/overnight bets where fewer humans are watching the market.
- **Observation mode** on adjacent markets — the bot watches but doesn't trade S&P 500 short-term contracts, weather-temperature markets across 19 U.S. cities, and live sports markets across 28 leagues. These are research feeds: it's learning what works without risking money.

A few engines that used to be live have been turned off — like the hourly crypto markets (turned off in April after correlated losses) and the weather "no" side (turned off in May after it became clear the edge wasn't real). Turning things off is part of the discipline.

The part that might be more important than the bot

Here's the thing I'm most excited about, and the part that's least obvious from the outside.

Every WebSocket message that the bot receives from Kalshi — every tick of the orderbook, every trade, every small movement in every market — is being captured **byte-for-byte** and saved permanently to a cloud archive. This started running in production on May 17, 2026.

This sounds boring. It is not boring.

Nobody else has this data. Kalshi does not sell historical raw orderbook streams. The exchange itself only keeps recent data live; what flowed across the wire on a Tuesday afternoon last March is gone from their public infrastructure once the markets settle. If you didn't capture it as it happened, it's gone forever.

Every day adds to it. A day of capture costs about a dollar or two in cloud storage. After a year, I'll have something genuinely rare: a complete, microsecond-resolution recording of every market Kalshi runs. After three years, I'll have something a small hedge fund would have to pay millions to assemble — except they couldn't, because most of the days they'd need are already in the past, and the data is gone.

It compounds without effort. The trading bot fights to keep its edge — every month, more sophisticated traders show up on Kalshi, and the easy mistakes the market used to make get smaller. The data archive has no such enemy. It just grows.

It enables the next bot, and the one after that. Any model I want to build six months from now — a different sport, a new market, a new probability technique — gets to train and test against the full historical record. Without the archive, I'd be limited to whatever data exists when I start. With it, I get to time-travel.

The honest first-principles framing is: *the trading bot is the proof-of-concept and the cash flow. The data corpus might be the bigger asset.*

Why this is only possible now

I’m one person. I don’t have a team. The bot has hundreds of thousands of lines of code, dozens of tests for every component, automated deploys, around-the-clock monitoring, and a research process that catches its own mistakes. A traditional engineering team to build this would be somewhere between five and fifteen people.

The answer is AI agents. I work with Claude — Anthropic’s AI assistant — many hours every day. The agent doesn’t just “write code.” It reads the entire codebase, proposes plans, writes tests *before* writing the code (so I can verify the test catches the bug), implements the change, runs the test suite to confirm nothing else broke, and presents the result. When something is sufficiently risky, I have *another* AI agent review the work adversarially — looking for everything that could be wrong — and I don’t ship until that adversarial reviewer says zero critical issues, twice in a row.

This pattern — one human directing many AI agents, each doing focused work with verification — is genuinely new. It’s the reason a one-person operation can run a system this complex without it collapsing into spaghetti and bugs. It’s also the reason I think this project is durable: the more carefully you structure how the agents work, the faster you can go and the less you break, which compounds the same way the data does.

The project is structured around this. There are formal “pillars” — paved roads — that the AI agents are required to follow: write the failing test first, run the equivalence test before changing anything load-bearing, get an adversarial review on risky changes, never bypass safety checks. They sound bureaucratic. They are the reason I sleep at night.

What could go wrong (the honest version)

I’d rather you hear this from me than figure it out yourself.

The model edge can shrink. As Kalshi matures and more sophisticated traders show up, the mistakes the market used to make get smaller. The bot’s edge is competitive — it has to keep improving to stay ahead. Some of the techniques that worked six months ago are already less profitable today. The plan is that the data corpus + agentic-engineering velocity lets me keep finding the next edge faster than the old ones decay. That’s the plan; it’s not a guarantee.

Markets break in weird ways. A data feed went down, an exchange returned a 403 error for hours, a price source stopped updating mid-trade. I’ve had each of these happen. Each one taught me something and got patched. There will be more.

Operational risk. The bot lives on one small cloud server. If it crashes at the wrong moment, an order can get stuck. The system has crash-recovery and watchdogs, but “no operational losses ever” is not a real promise anyone can make about software.

Concentration. This is one person, on one bot, on one exchange, in one regulatory regime. A wide variety of bad days are possible.

The honest probability of any of this destroying the project. Low for any single one, real for the combined set. The way I manage it is small bet sizes, automatic drawdown protection, kill-switches that I or the bot can pull instantly, complete logs of every decision, and a corpus of raw data that — even if the bot itself dies tomorrow — keeps accumulating value that someone (me, or a future buyer) can use.

Why I'm telling you all this

If you're family or a friend reading this, the short version is: I'm building something serious, I'm being conservative with risk, I'm honest about what I don't know, and I find it the most interesting work I've ever done. The robot trades. The recording of everything it sees might end up being worth more than the trading. And it's only possible because of the AI tools that came out in the last couple of years, which let one person with care and discipline do what used to take a team.

If you want the technical version — the math, the architecture, every layer of every model — there's a separate technical whitepaper that covers it. If you want the version meant for someone considering investing capital, there's an investor whitepaper. This one is just to give you a real picture of what's actually going on.

Happy to answer anything.

— Gabe